

Applied Industrial Organization

Stata Lecture 1

Gianluca Mazzarella

February-May 2026

1 Introduction

1.1 What is Stata

- Stata is a Statistical Software owned by the *StataCorp*.
- The first Stata version was released in 1985.
- Currently, Stata 19 is the last stable release. This version has been launched in April 2025. Usually, a new version is released once every two years.
- StataCorp releases three different “editions” of the software:
 1. **BE:** *Basic Edition*
 2. **SE:** *Standard Edition*
 3. **MP:** *Multi-Processors* for parallel computing.
- StataCorp manages also *The Stata Journal* in which are published all the new commands provided within Stata.¹

1.2 Pro and Cons

Stata is probably the most used software in (Micro-)Economics and Social Sciences. The main advantages are (according to my opinion):

- Relatively easy syntax.
- It is probably the easiest software to “explore” data that you do not know.
- Relatively good documentation.
- Great variety of tools for data manipulation

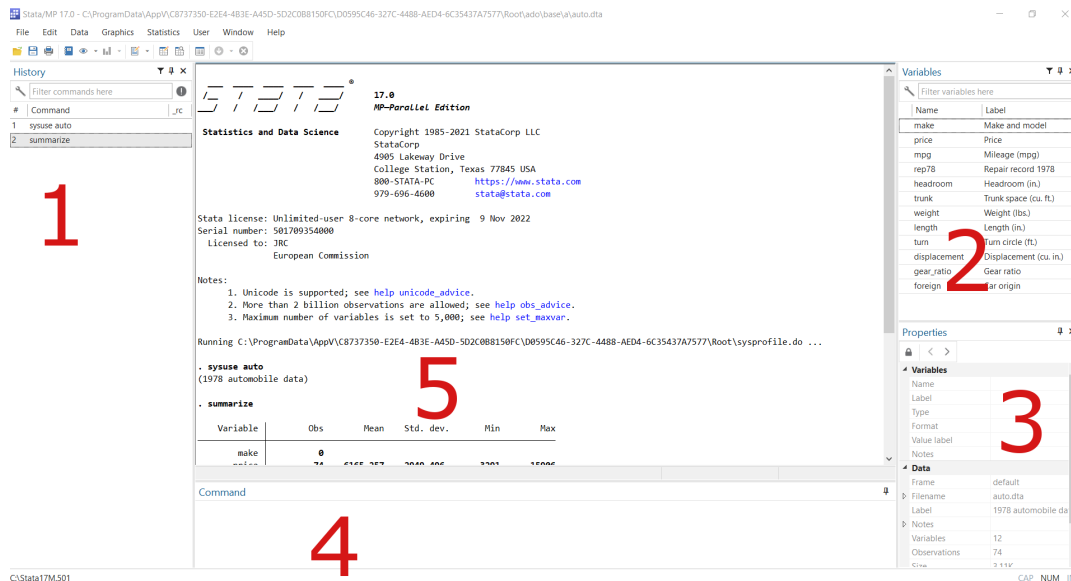
¹See: <https://journals.sagepub.com/home/stj>

- Most of the econometrics methods are already implemented
- Easy to work with “dates”

The main disadvantages are (still according to my opinion):

- It is not an object-oriented language (such as Python, R, C++, etc.). This basically means that you can work with a database at the same time.
- Not the best software for producing graphs.
- Not the easiest software for developing new statistical-econometrics methods.
- It is not an open-source software.

1.3 How it looks like



1. In the box named *History*, you can find the last launched commands.
2. In the box named *Variables*, you can find the list of variables contained in the current database.
3. In the box named *Properties*, you can find the description of the current database.
4. In the box named *Command*, you can type commands to be executed (not a good practice!).
5. Finally, in the central part of the windows, we can see the executed commands and the produced results.

1.4 Files

Stata mainly works with the following files:

- File with extension `.dta`. These files contain databases.
- File with extension `.do`. These files are usually called *dofiles*. They are basically text files that contain a list of commands that can be executed in Stata.
- File with extension `.log`. These are *log files*. These files contain both the commands and their results. We will discuss them in a while.
- File with extension `.ado`. These files contain *programs*, i.e. you can define your own commands in the ado files. **[Not in scope of this course!]**

What a database is.

In computing, a database is an organized collection of data stored and accessed electronically. [source: Wikipedia]

- A “simple” way to define a database is like a matrix with N rows and K columns.
- The rows are the *observations*. They can be individuals, cities, firms, etc.
- The columns are the available information. They are usually called *variables*. Each variable is identified by a name (variable name). In Stata, the variable name cannot contain spaces and cannot start with numbers (e.g. `population2011` and `population_2011` are allowed, `2011population` and `population 2011` are not).
- Variables can be *numerical*, such as population, income, gross value added, etc. (in principle there is no distinction in Stata between *continuous* and *categorical* variables), or *characters* (strings), such as name of the city, job description, NACE code, etc.
- The combination of n and k is usually called *cell* and it represent the information k for the individual n .

2 Basic Commands

General structure of the commands

In Stata, typically all the commands follow this structure:

```
name_of_the_command arguments, options
```

where *arguments* can be the name of the database, a list of variables, etc., while *options* are all the specific options of the command.

- **Help:** If you do not remember how a command works or which options can be specified, you can type:

```
help name_of_the_command
```



- **Some basic computation:** Stata is a strange software! If you try to type in your command line something like $2+2$, it will give you an error:

```
2+2
2 is not a valid command name
r(199);
```

This is because Stata always expects the command name as the first word. So, if you want to see the result of some basic operation (or simply to see as output some numbers of characters), you have to use the command `display`:

```
display 2+2
4
display 2
2
display "hello"
hello
```

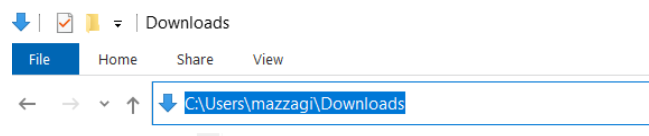
- **Set working directory:** in the bottom-left corner, you can see which is your current working directory. It is fundamental to set the working directory because it is there that Stata will search all the files that it needs. To change the working directory type for example:

```
cd C:/Users/gianluca/Dropbox/Lect1
```

If your path includes spaces, you need to insert it between commas:

```
cd "C:/Users/gianluca-mazzarella/Dropbox/Lect1"
```

In Windows OS, you can copy-paste the path using the file explorer:



Note that in Windows OS, paths can be specified both with the symbol `\` and with the symbol `/`:

```
cd C:/Users/gianluca/Dropbox/Lect1
cd C:\Users\gianluca\Dropbox\Lect1
cd C:/Users/gianluca\Dropbox\Lect1
```

while paths in Mac OS and Linux OS need to be specified using `/`:

```
cd /home/gianluca/Dropbox/Lect1
```

Note that if you simply type `cd` without any path, in Windows OS, Stata will print the current working directory, while in Mac OS, Stata will set as working directory the default one (usually something like `"/home/user"`).

- **Load a database:** A database can be loaded using the command

```
use database_name
```

Note that it is not necessary to specify the extension `.dta` in the file name. If you have already loaded another database and you have some *unsaved changes* on it and you try to load another database, Stata will give you the following error:

```
use database2
no; dataset in memory has changed since last saved
r(4);
```

In order to avoid it, you can specify the option `clear` or the option `replace`:

```
use database2, clear
use database2, replace
```

Alternatively, you can use the command `clear` *before* the command `use`:

```
clear
use database2
```

Remember that Stata will search the database in your current working directory.

If in your working directory, the database that you are trying to load does not exist, Stata will give you the following error:

```
use database
file database.dta not found
r(601);
```

- **Save a database:** A database can be saved using the command

```
save database_name
```

Also, in this case, it is not necessary to specify the extension `.dta` in the file name. If in your working directory, a database with the same name already exists, Stata will give you the following error:

```
save database_name
file database_name.dta already exists
r(602);
```

In order to avoid it, you can specify the option `replace`:

```
save database_name, replace
```

Pay attention every time you use this option!

- **See the database:** If you want to have a look at your database, you can use the command:

```
browse
```

If you want to modify it, you can use the command:

```
edit
```

PLEASE DO NOT USE IT!!

- **Count the number of observations:** If you need to know how many observations there are in your database, you can use the command:

```
count
```

- **Describe the database:** If you want to have a general description of the variables contained in your database, you can use the command:

```
describe
```

In the output, you can see the list of variables, whether they are string (“Storage type” that starts with `str` followed by the length of the string) or numerical (`byte`, `int`, `long`, `float` or `double`) plus other information that we will discuss in the next lessons.

If you want to see the description of a subset of variables, you can also specify a variable-list after the command:

```
describe var1 var2 var3
```

Specify more than one variable

- In general, if you need to specify more than one variable, you can insert them one by one separated by a comma (e.g. `var1 var2 var3`).
- If you have to specify a range of variables, you can insert the first and the last one separated by “-” (e.g. `var1-var10`)
- If you have to include all the variables with a certain path, you can use the character “*” in the following way:

- * “`var*`”: all the variables that *start* with “`var`”
- * “`*suff`”: all the variables that *end* with “`suff`”
- * “`*path*`”: all the variables that *contain* “`path`”

Note that Stata is *case-sensitive*. This means that `var1` $\neq$ `Var1` $\neq$ `VAR1`

- **Print summary statistics:** if you want to have some descriptive statistics of each variable of your database, you can use the command:

```
summarize
```

Also, in this case, you can specify a subset of variables, by doing:

```
summarize var1 var2 var3
```

Exercise

1. Have a look with the help of the command `describe`
2. Look at its first option
3. Import with the command `webuse` the database named *lifeexp*.
4. Summarize the variable *safewater* using this option.
5. Look at the output

Abbreviations in Stata

- It is very common in Stata to *abbreviate* commands.
- For example:
`use database`
can be abbreviated in
`u database`
- You can understand which are the *compulsory letters* by looking at the help of each command. These are the ones *underlined* in the syntax description.
- **Suggestion:**
 1. Use abbreviations that you can understand!
 2. Try to be internally *consistent*!

Exercise

See how the commands `browse`, `save`, `count`, `describe` and `summarize` can be abbreviated.

- **Log files:** It is possible to save the executed codes and their outputs in a text file named *log file*. They work in this way:

1. Create a log file using the command:

```
log using filename.log
```

Note that you can specify the option `replace` if the file already exists and you want to overwrite it or the option `append` if the file already exists and you want to add additional codes to it.

2. Run a series of commands.
3. Close the log file using:

```
log close
```

Note that the log file supports (among others) the options **replace** and **append**. The first one will overwrite (if any) the existing log file with the same name; the second will include the new command at the bottom part of the log-file.

Exercise

1. Set your working directory
2. Open a log file named *yourname_yoursurname.log*
3. Re-run the command at point 1 in order to include it in the log file.
4. Load the database named *data_vaccines.dta*
5. Describe the content of the database
6. Summarise (in one line) all the numerical variables (you can choose how to select those variables)
7. Summarise with details the first numerical, not categorical(!) variable
8. Display its minimum value (type it in the command line looking at the output)
9. Close the log file

- **Tabulation:** if you have some categorical variable (numerical or character) and you want to see the frequency distribution, you can use the command:

```
tabulate varname  
ta varname
```

The command `tabulate` also allows us to see the cross-tabulation of two variables, by doing:

```
ta varname1 varname2
```

Exercise

Have a look to the options of the command **tabulate**.

3 Do-files

The best way to use Stata is by including all the codes that need to be executed in a text file named “*do-file*”. **Why you should use do-files instead of directly writing**

your code in the command line:

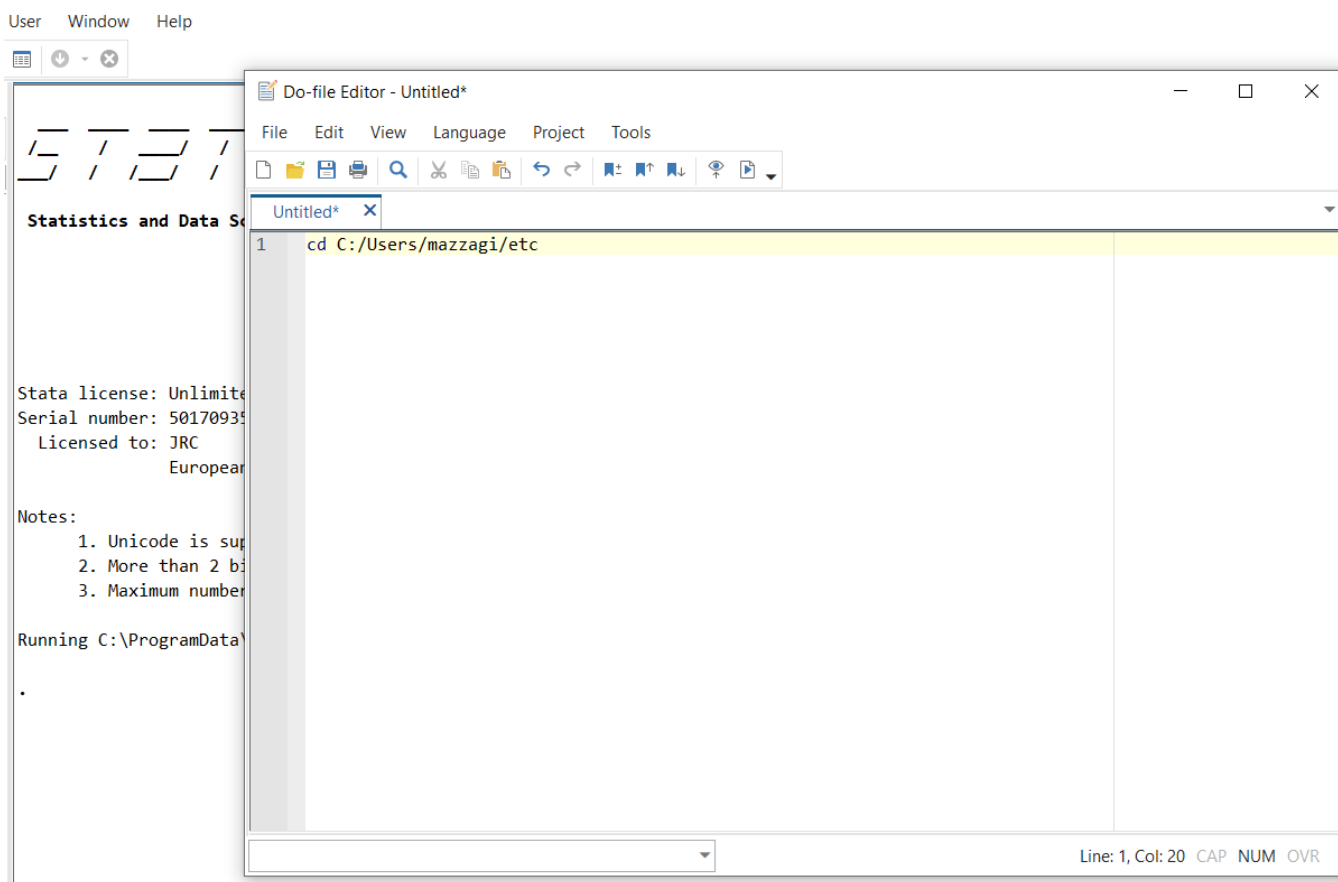
The main reason is *replicability*! In particular:

- If you directly code using the command line, you will keep track of what you are doing only until you close your current Stata session.
- It may happen that you will need to change something in your analysis or an update of your database has been released, so you have to repeat your list of codes.
- You work with someone else (e.g. a colleague, a coauthor) and you need to integrate something in the analysis done by her/him.
- You need to publish an article/report and the journal wants all the codes used in your analysis.

For this reason, you have to be able to keep track of what you have done even months after and the easiest way of doing it is to keep all the commands in a do-file.

3.1 Features of a dofile

- A do-file has the extension `.do`
- They are in a separate window with respect to the main Stata window



- The do-file window can be opened using the shortcut Ctrl + 9 (cmd + 9 on Mac OS) or by clicking on Windows –> Do-file Editor –> New Do-file Editor
- Can be saved using the Command Ctrl + S
- The entire do-file can be executed using the shortcut Ctrl + D (DO). Similarly, if you highlight some rows of your do-file and then use the same shortcut (Ctrl + D), Stata will execute only the highlighted rows.
- If you double-click on a file with the .do extension, a new Stata session will be initiated.

(My) **general rule:** include in the do-file all the commands that modify the database to keep track of all the actions. While, if you need to *explore* the dataset (e.g. doing some descriptive statistics, summary, tabulation, etc.), you can directly use the command line.

3.2 Comments

It is important to keep track of what you are doing *within the do-file*. For this reason, it is fundamental to **comment** the do-file. In Stata, you can insert comments in the do-file in three different ways:

1. If you need to comment on an entire line, you can use the character `*` at the beginning of the line

```
* here a comment  
use database , clear
```

2. If you want to add a comment *after* a command line you can use the characters `//`

```
use database , clear // I load data about something
```

3. If you want to comment *multiple lines* you can type `/*` at the beginning of the block that needs to be commented and `*/` at the end.

```
/* In this do-file I want to do the following:  
1. Create this data  
2. Something else  
*/  
use database , clear
```

3.3 General rules for a clear do-file

Some general suggestions for a clear do-file:

1. Try not to create do-files that want to do too many things. For example, keep separate the do-files that clean your data from the ones that do the analysis on the final database.
2. Give to the do-files clear names. By doing so, it will be much easier to find after some months the do-file that does certain operations. For example, you can start the name of the do-file with a progressive number that indicates in which order they have to be executed (e.g. `1_cleaning.do`, `2_analysis.do`)
3. Comment as much as possible.
4. Put a blank line between blocks of codes that do different operations.
5. Indent when necessary (we will discuss it next lessons)

A good structure of a do-file for data-cleaning should be the following:

1. Start the do-file with 2-3 lines of comments in which you explain:
 - The day on which you started the creation of the do-file
 - The general aim of the do-file (e.g. starting from these databases, create a database that do...)
 - (hopefully) The necessary steps for doing this.
2. Then set the working directory

3. Then load the necessary database
4. Do all the necessary operations on it
5. Save the final database

4 Modify a dataset – Basic operations

- **Generate a new variable:** If you need to create a new variable, you have to use the command:

```
generate newvar = something  
ge newvar = something  
g newvar = something
```

The main operations are:

– Plus

```
ge newvar = var1 + var2  
ge newvar = 5 + var1
```

– Subtraction

```
ge newvar = var1 - var2  
ge newvar = var1 - 2
```

– Multiplication

```
ge newvar = var1 * var2  
ge newvar = var1 * 0.5
```

– Division

```
ge newvar = var1/var2  
ge newvar = var1/2
```

– Exponentiation (e.g x^2 , x^3 , $x^{1/2}$, ...)

```
ge newvar = var1^2  
ge newvar = var1^(1/2)  
ge newvar = var1^.5
```

– Logaritms

```
ge newvar = log(var1)
```

– Square-root

```
ge newvar = sqrt(var1)  
ge newvar = var1^(1/2)
```

- **Exponential**

```
ge newvar = exp(var1)
```

Some notes on the variable creation:

- Obviously, you can combine all the functions above using parentheses:

```
ge newvar = sqrt(exp(var1 + 1) - 56 * 3)
```

Remember that Stata executes the operations following the same order of the algebraic expressions (first $\wedge$, then $*$ and $/$, finally $+$ and $-$).

Pay attention on it and ever verify that the result is what you expected!

- If you try to generate a new variable by giving it the name of a variable already present in the database, Stata will give you an error:

```
ge var1 = var1*2
variable var1 already defined
r(110);
```

- Modify the content of an existing variable:

```
replace var1 = var1*2
```

- Drop a variable:

```
drop var1
drop var1 var2
drop var1-var10
```

- On the contrary, **keep only some variables**:

```
keep var1
keep var1 var2
keep var1-var10
```

- Rename a variable

```
rename oldvarname newvarname
```

5 Conditions

You may often want to perform operations that involve only a subset of observations (e.g. summarize, browse or count a subset of observations, generate a variable that takes some values if a certain condition is verified and other values if not). In this case, you can specify a *condition* using the command `if` between the arguments and the options of the

command.

In my opinion, the best way to think about conditions is to think about expressions that return only the values **true** and **false**.

How to specify conditions:

- Equal to: `var1 == 2` or `var1 == var2` (note that in the conditions the equality is expressed using two equal symbols)
- Different to: `var1 != 2`. In general the symbol `!` means *not* and reverses the conditions. For example `var1 < var2` is equivalent to `!(var1 >= var2)`
- `var1 > var2` (greater) `var1 >= var2` (greater or equal)
- `var1 < var2` (smaller) `var1 <= var2` (smaller or equal)

If you need to specify multiple conditions are particularly important the operators `&` (and) and `|` (or). The first one returns true if **both** are true and false in all the other cases, while the operator *or* returns true if **at least one condition** is true and false if all the conditions are false. For example:

```
cou if var1>=0 & var1 < 3
cou if var1>=0 | var2>var1
count if (var1<=-2 | var1 >=2) & var2>var1
```

You can also specify conditions on string variables. The conditions can be only *equal to* or *different to*. For example:

```
cou if var1=="Lombardia"
cou if var1 != "Rest of the World"
cou if var1 != "Rest of the World" & var2>=0
```

For example, if you need the absolute value of a variable:

```
ge newvar = var1 if var1>=0
replace newvar = -var1 if var1<0
```

You can also directly generate *dummy* variables, providing conditions as arguments. For example:

```
ge negative = var1<0
ge female = gender == "F"
```

The first variable takes the value 1 if `var1 < 0` and 0 otherwise. The second takes the value 1 if gender is equal to "F" and 0 otherwise.

Conditions can also be used to reduce your database to a subsample that has certain characteristics. You can do this with the commands **keep if** or **drop if**:

```
keep if gender == "F"  
drop if var1<0  
keep if gender == "F" & var1>=0
```


Applied Industrial Organization

Lect 2 – Across rows operations

Gianluca Mazzarella

February-May 2026

1 Sort a database

You can easily *rearrange* a database by sorting it by some characteristics (both numerical and/or characters). You can do this with the command `sort`

```
sort district
sort year
sort district year
```

If you specify more than one variable, Stata will sort first by the first specified variable, then, within the same value of the first variable, by the second, etc.

In principle Stata uses the *ascending order*, so from the smallest to the largest for numerical variables and from **A** to **Z** for character variables. It is also possible to do the opposite with the command `gsort` by inserting a “-” before the variable for which you want to sort in descending order:

```
gsort -year
gsort district -year
```

Note that in the second case, Stata sorts first by district in *ascending order* and then by year in *descending order*.

Note also that the last row of the command `describe` tells you if the database is already sorted.

2 Operations across rows

All the operations seen so far are about computing something *within* row. You may often want to compute some operations *across* rows. For example, you may want to compute the deviation from the mean of a certain variable and so, you first need to compute the mean of that variable. You can do this with the command `egen` which means *extensions to generate*. So in the example above:

```
egen pop_mean = mean(pop_tot)
```

```
ge pop_deviations = pop_tot - pop_mean
drop pop_mean
```

The most important functions associated to the `egen` command are:

```
egen var1 = mean(pop_tot) // mean
egen var2 = median(pop_tot) // median
egen var3 = min(pop_tot) // min
egen var4 = max(pop_tot) // max
egen var5 = pctlile(pop_tot), p(75) // to compute percentiles
egen var6 = total(pop_tot) // to compute the sum of the variable
```

You can find the full list of functions supported by `egen` by typing `help egen`.

Another useful command is the command `xtile` which categorizes a continuous variable according to its quantiles:

```
xtile var6 = pop_tot, nq(4)
```

In the example above, the variable `var6` takes values:

$$\text{var6} = \begin{cases} 1 = \text{below 25\% percentile} \\ 2 = \text{between 25\% percentile and 50\% percentile} \\ 3 = \text{between 50\% percentile and 75\% percentile} \\ 4 = \text{above 75\% percentile} \end{cases}$$

Other two important functions are `_n` and `_N` that return the current observation and the total number of observations respectively:

```
ge num_obs = _n
ge tot_obs = _N
```

3 Operations across rows and *within group*

It is quite common to compute operations as the ones described in the previous section (e.g. mean, min, max, etc.) within some specified group. For example, you want to compute the mean of the population *across* district, but *within* year. You can do this with the operator `by` before the command. For example:

```
by year: egen mean_pop = mean(pop_tot)
```

There is one important thing to consider: Stata wants that the database is *sorted* by the variable(s) that you are using as an argument of the command `by`, otherwise it will give you the following error:

```
by year: egen mean_pop = mean(pop_tot)
not sorted
r(5);
```

You can address this issue in 2 ways:

1. You can obviously sort the database before the command `by`:

```
sort year
by year: egen mean_pop = mean(pop_tot)
```

2. You can use the command `bysort`:

```
bysort year: egen mean_pop = mean(pop_tot)
bys year: egen mean_pop = mean(pop_tot)
```

Note that there are operations in which you have to sort by including more characteristics than the ones that you need to include in the command `by`. For example, you want to compute a variable that takes value 1 if, in a certain year, this district has the highest population, 2 if it has the second highest, etc. To do this you have to do:

```
gsort year -pop_tot
by year: ge rank_pop = _n
```

In these cases, pay particular attention to not using the command `bysort`, otherwise, you will sort the database again without taking into account the population.

You can also *summarize* certain characteristics of the database by *collapsing* the database `by` some variables and providing some statistics that need to be computed. For example, you want to create a database that contains the average population by year. You can do this with the command `collapse`:

```
collapse (mean) pop_tot , by(year)
```

Note that Stata returns a database with only the variables specified as arguments and the `by` variables. You can obviously specify more than one variable at a time:

```
collapse (mean) pop_tot pop_males pop_females , by(year)
collapse (mean) pop_tot (sum) pop_males (p50) pop_females , by(year)
```

If you want to compute more than one statistic for the same variable, you need to specify a variable name:

```
collapse (mean) mean_pop=pop_tot (sum) sum_pop = pop_tot , by(year)
```

4 Some random stuff

4.1 Tabulation

If you have some categorical variable (numerical of character) and you want to see the frequency distribution, you can use the command:

```
tabulate varname
ta varname
```

The command `tabulate` also allows us to see the cross-tabulation of two variables, by doing:

```
ta varname1 varname2
```

Exercise

Have a look to the options of the command `tabulate`.

4.2 Missing Values

It is quite common that for some observations you cannot observe the content of some cells. These values are usually called *missing values*. Stata records missing values with a **dot** (.) for numeric variables and with an **empty string** for character variables. These values are particularly useful for example in conditions:

```
ge female = 1 if sex == "F"  
replace female = 0 if female ==.
```

```
ge sex = "F" if female == 1  
replace sex = "M" if sex == ""
```

Note the following things:

1. In conditions, it makes sense only to express conditions of equal and not equal on missing values.
2. All the software expresses missing values as +Infinity or -Infinity. So, pay attention when you have conditions such as:

```
ge dummy = 1 if var1 >= 0
```

if `var1` contains missings. It is much better to explicitly include the missing condition:

```
ge dummy = 1 if var1 >= 0 & var1 !=.  
replace dummy = 0 if var1 < 0 & var1 !=.
```

4.3 Commands in more than one line

Usually Stata follows the rule “*one line-one command*”. This means that Stata assumes that the end of the line coincides with the end of the command. If you need more “space” to complete the arguments of a command, you can use the following two alternatives:

1. **Line breaks:** if at the end of a line you insert the following sequence: `///`. In this way, Stata understands that the command is not completed.

```
replace var1 = 0 if var1 < 0 | var2 < 0 | ///  
var3 < 0 | var4 < 0
```

2. In alternative, you can say to Stata that from a certain point, it has to consider a command completed when it finds a semicolon. You can do this by typing:

```
# delimit ;
histogram cluster_dimension , discrete width(1) frequency
    fcolor(navy*.7) lcolor(navy)
    ytitle("Number-of-Municipalities", size(vsmall) box fcolor(none)lcolor(none))
    ylabel(,labsize(vsmall)) xtitle(Cluster Size)
    xtitle(,size(vsmall)) xline(5, lpattern(dash)) xlabel(2(1)28, labsize(vsmall))
    graphregion(c(white))
;
# delimit cr
```

Note that first, you type `# delimit ;` and from this point, Stata “waits” to find a semicolon in order to consider a command complete. Then, when you type `# delimit cr` you return to the “standard mode”.

4.4 Preserve & Restore

Sometimes, it happens that you want to change your database, but you do not want these changes to be *permanent*. For example, you may want to plot the average population by year. So you need to **collapse** your database in order to compute the means, but then (after the plot) you want to return to the original database. You can do this with the commands **preserve** and **restore**:

```
preserve  
    collapse (mean) pop_tot , by(year)  
    line pop_tot year  
restore
```

Or for example:

```
preserve  
    keep if gender == "F"  
    drop gender  
    save only_females , replace  
restore
```

4.5 Indentation

In basically all the programming languages, when a line is *directly related*, e.g. as in the previous case, the line is a continuation of the line above, it is a good practice to *indent* that line. You can do this with the **tab** key on your keyboard.

Why is it important to properly indent in the do-file? Mainly because this helps you to understand the main “dependencies” of this line. For example:

```
use ${our_dir}/temp/icc ,clear
forvalues bw = 5(2.5)10{
noi: di 'bw'
matrix res =J(100,4,.)

forvalues d = 1(1)100 {
matrix res['d',1]='d'
loneway target_var group_var if ///
cond_var>=('d'-'bw')*10 & ///
cond_var<=('d'+'bw')*10
matrix res['d',2]=r(rho)
matrix res['d',3]=r(se)
matrix res['d',4]=r(N)
}
preserve
clear
svmat res
rename res1 cent
rename res2 icc
rename res3 se_icc
rename res4 num_icc
ge bw = 'bw'
if ('bw'!=5){
append using toappend
}
save toappend, replace
restore
}
use toappend, clear
save ${our_dir}/temp/stime ,replace
```

Compared to:

```
use ${our_dir}/temp/icc , clear
forvalues bw = 5(2.5)10{
    noi: di 'bw'
    matrix res =J(100,4,.)

    forvalues d = 1(1)100 {
        matrix res['d',1]='d'
        loneway target_var group_var if ///
            cond_var>=('d'-'bw')*10 & ///
            cond_var<=('d'+'bw')*10
        matrix res['d',2]=r(rho)
        matrix res['d',3]=r(se)
        matrix res['d',4]=r(N)
    }
    preserve
    clear
    svmat res
    rename res1 cent
    rename res2 icc
    rename res3 se_icc
    rename res4 num_icc
    ge bw = 'bw'
    if ('bw'!=5){
        append using toappend
    }
    save toappend, replace
    restore
}
use toappend, clear
save ${our_dir}/temp/stime , replace
```

Pay particular attention to the indentation in the following situations:

1. Loops (foreach & forvalues).[later in this course]
2. Preserve & Restore [see above]
3. Commands in multiple lines (see above).

4.6 Importing external data in Stata

Data are not “born” in a Stata format, so usually, you have to import them. The two most common data formats that you may find are:

1. **Excel files:** you can import Excel files directly within Stata with the command:


```
import excel using excelfile.xlsx
import excel using excelfile.xls
```

As for the command `use`, if you already have loaded another dataset you have to specify the option `clear`:

```
import excel using excelfile.xlsx , clear
```

If the first row of the Excel file contains the variable names, you can use the option:

```
import excel using excelfile.xlsx , firstrow
```

2. **Delimited files:** it is quite common that data are shared in a format in which variables are delimited by a pre-defined character, such as commas, semicolons, tabs, etc. The most common extensions of these files are `.csv` and `.txt`. You can import them with the command:

```
import delimited using filename.csv
import delimited using filename.csv , clear varnames(1)
```

By default, Stata assumes that variables are delimited by a comma. If it is not so, you have to specify the option:

```
import delimited using filename.csv , delim(";")
```

3. **Other software:** starting from version 15, Stata also allows to directly import data from some different statistical software, for example:

```
import sas using databaseinsas.sas7bdat
import sas using databasename.sas7bdat , clear
```

for database written using SAS (<https://www.sas.com/>), or:

```
import spss using databaseinspss.sav
import spss using databasename.sav , clear
```

for database written using SPSS (<https://www.ibm.com/spss>).

You can find the full list of extensions supported by Stata, by typing `help import`.

Applied Industrial Organization

Lect 3 – Graphs in Stata

Gianluca Mazzarella

February-May 2026

1 Producing graphs with Stata

Histograms

If you are interested in the distribution of a *numerical* variable, you can use the command:

```
histogram variable  
hist variable
```

The command supports the following (main) options:

- **freq** if you want to plot *frequencies* instead of density
- **discrete** if the variable is *discrete*
- **xline(k)** if you want to add a vertical line at *k*.
- **bin(k)**, **width(k)** and **start(k)** if you want to specify the number (**bin**) or the size (**width**) of the bins and/or the start value (**start**)

The command **hist** supports (as all the graphs in Stata) the selection of a subsample of observations using the command **if**:

```
hist variable if gender == "F", discrete
```

Pies

However, if you need to plot the distribution of a *categorical* variable, there are solutions that are more “appropriate”. For example:

```
graph pie , over(gender)
```

Note that in this case, the variable **gender** is included as an argument of the option “over”.

You can also produce pies by including a variable in the form:

```
graph pie var1 , over(gender)
```

In this way, Stata will sum the content of the variable `var1` for each value of the variable `gender` and it will plot the proportion for each category.

Another option is to not include the option `over` and to specify more than one variable as an argument:

```
graph pie var1 var2 var3
```

In this way, Stata will sum the content of each variable and will plot the proportion of each variable with respect to the sum of all variables.

Bars

Instead of drawing *pies*, you can also plot *bars*, using the command:

```
graph bar (count), over(gender)
graph hbar (count), over(gender)
```

Note that the syntax is quite similar to `graph pie`, with the exception of the argument `(count)`. Why? Because with the command `graph bar` you can plot a series of statistics of some variables broken by the categorical variable contained in the option `over`. For example:

```
graph bar (mean) income, over(gender)
ge foreign = nationality != "Italian"
graph bar (sum) foreign, over(gender)
```

The command also supports multiple over variables:

```
graph bar (mean) income, over(gender) over(nationality)
```

Note that in this case, as in many other cases, the option `over` supports its own *sub-options*. For example:

```
graph bar (count), over(district, label(labsize(small) angle(45)))
```

Boxes

Another way to look at the distribution of a variable is to plot the so-called *boxplots*. You can do this in Stata with the command:

```
graph box income
graph box income if gender == "F"
```

You can also plot the distribution *over* some categorical variables:

```
graph box income, over(gender)
```

Relation between continuous variables

Scatter

If you want to investigate the relation between two continuous variables, you can use the command:

```
scatter income wealth  
scatter income wealth, connect(line)
```

Line

Instead of drawing dots, you can also plot lines:

```
line income wage
```

Remember to sort before for the X variable or to specify the option **sort**.

How to export graphs

In general, you do not produce graphs to look at them on the screen, but you want to include them in a report/paper/thesis, etc.

If you want to export a graph in a file you can use the command:

```
graph export name_file.png
```

Note the following things:

1. Stata automatically takes the extensions from the one specified after the file name.
2. You can use as extensions (the most common): .jpg, .pdf, .png, .eps.
3. graph export follows the same rules of **save**, so if the file already exists, Stata will give you an error:

```
graph export name_file.png  
file name_file.png already exists  
r(602);
```

Also, in this case, you can specify the option:

```
graph export name_file.png, replace
```

graph display

Automatically, Stata will export the graph using the ratios of the current window. If you want to resize it, you can use the command:

```
graph display, ysize(10) xsize(10)
```

In this way, with the options **ysize** and **xsize**, you can alter this ratio and then export it.

graph combine

You can also put in the same window more than one graph. In order to do this, you first need to *temporarily save* each graph, and then combine them. You can temporarily save a graph with the option:

```
scatter income wealth , name(graph1)  
graph box income , name(graph2)
```

And then, combine them with the command:

```
graph combine graph1 graph2  
graph combine graph1 graph2 , rows(2)  
graph combine graph1 graph2 , cols(1)
```

Remember to drop after the **graph combine** the graphs temporarily saved by typing:

```
graph drop graph1 graph2  
graph drop _all
```

Applied Industrial Organization

Lect 4

Gianluca Mazzarella

February-May 2026

1 Additional objects & post-estimation commands

As discussed in the first lecture, Stata works with one database at a time, however, you can use *additional objects* separated from the used database. These are:

- **Scalars:** these are numeric uni-dimensional objects. For example:

```
scalar s1 = 5
display s1
scalar s2 = 2+2
display s2
```

- **Locals:** these objects can be both numeric or characters and they *disappear* when the execution of the do-file is concluded. They can be called including their name in these two characters: ‘ and ’

```
local loc1 = 5
display ‘loc1’
local loc2 = "wages"
display "‘loc2’"
```

- **Globals:** these are quite similar to the locals, but they remain in memory and they are not deleted when the execution is concluded. They can be called using the character \$ before their name

```
global g1 = 5
display $g1
global g2 = "wages"
display "$g2"
```

- **Matrices:** these are numerical matrices.

```

matrix A = ( 1, 2 \ 3, 4 )
matrix list A
matrix A[1,1] = 10
matrix list A
display A[1,1]

```

As you may note the following things:

- Each operation on a matrix starts with the command **matrix**.
- You can define a new matrix by inserting its elements *by row*, separating each element by a comma and each row by a backslash.
- You can see the content of a matrix by writing **matrix list** and then the name of a matrix
- You can refer to a specific element of a matrix indicating its position within square brackets.

1.1 Post estimation commands

Basically all the commands that do not modify the database (e.g. generate, replace, drop, keep, etc.) automatically produce a list of *post-estimation* values. You can see the entire list by typing:

```

su
return list

```

All these post-estimation values are some objects like the ones described above (scalars, globals, matrices). These objects can be used in several operations. For example:

```

su pop_tot
local mean_pop = r(mean)
local obs = r(N)
hist pop_tot, xline( 'mean_pop' ) note( "Number of observations: - 'obs' " )

```

Note that this is for commands such as summarize, count, tabulate, etc. The *estimation commands* (such as regress, ivregress, probit, logit, etc.) support the function:

```

regress y x
ereturn list

```

2 How to organize a working directory

In general, it is not a good practice to save all the different files (original files, dta, do-files, graphs, etc.) in the same folder. An alternative can be to create a “main folder” and then create the following sub-folders:

1. **original_data**: for all the original databases;

2. **data**: for all the databases create;
3. **dofiles**: for do-files
4. **output**: for graphs, tables, etc.

In this way, you can set the working directory to the main folder and then, insert the name of the sub-folder directly in the command. For example:

```
cd cd "C:/Users/gianluca/One-Drive/AIO/lect5"
import delimited orig_data/pop2021.csv, clear
graph bar pop2021, over(country)
graph export output/graph_bar.png, replace
save data/pop2021, replace
```

In the alternative, in particular, if you have to structure your paths in a more complicated way, you can use globals:

```
cd "C:/Users/gianluca/One-Drive/AIO/lect5"

global orig_data "C:/Users/gianluca/One-Drive/pop_data"
global output "C:/Users/gianluca/One-Drive/report/graphs"
global data "C:/Users/gianluca/One-Drive/lect6/data"

import delimited "$orig_data/pop2021.csv", clear
graph bar pop2021, over(country)
graph export "$output/graph_bar.png", replace
save "$data/pop2021", replace
```

Suggestion: try at least to keep in separate folders:

- do-files
- data
- outputs (e.g. graphs & tables)

3 Automatize operations – loops

In informatics, quite often you have to repeat the same operation several times. For example, you have to compute the mean of several variables; or you have to append several files named in a similar way (e.g. pop2010, pop2011, etc). If so, you can save time and space by using loops.

In Stata, two kinds of loops exist: the **foreach** and the **forvalues**. Basically, the **forvalues** can be used when you have to move across a numeric argument, while the **foreach** can be used in more or less every circumstance. The syntax is the following:


```
foreach k in Male Female {
    preserve
    keep if sex == "'k'"
    save data_`k'
    restore
}
```

In particular:

- You want to repeat all the commands included between curly brackets for Male and Female. So two times.
- Each time the local **k** takes one of the values specified after **in**

Another example:

```
foreach k in pop_tot pop_males pop_females {
    egen mean_`k' = mean(`k')
}
```

The command **forvalues** follows instead the rules of numerical vector and the syntax is the following:

```
use filename1, clear
ge file_num = 1
forvalues i=2(1)10 {
    append using filename_`i'
    replace file_num = `i' if file_num == .
}
```

4 Convert numerical variables to character variables, the other way around & labels

As discussed in previous lectures, you may have 2 different types of variables:

1. Numerical variables, e.g. wages, population, etc.
2. Character variables, e.g. city of residence, etc.

You can distinguish between the two kinds of variables mainly in two ways:

1. By using the command **describe**
2. By looking at the variables in the **browse**: numerical variables are colored in *black*, while character variables in *red*.

You can convert character variables into numerical variables (if it is meaningful) in two ways:

```
ge numvar = real(charvar)
destring charvar, ge(numvar)
destring charvar, replace
```

On the contrary, you can convert a numerical variable into character variables by doing:

```
ge charvar = string(numvar)
tostring numvar, ge(charvar)
tostring numvar, replace
```

4.1 Labels

There is also something “in the middle”! You can have a numerical (categorical) variable in which each value that it can assume is associated with a *label*. An example can be the variable about the education attainment, for example:

Value	Label
0	No education
1	Primary
2	Lower Secondary
3	Higher Secondary
4	University Degree
5	PhD

In Stata, you can define *value label* using the following two commands:

1. First, you a to define a *label value*:

```
label define EDU 0 "No-education" 1 "Primary" ///
2 "Lower-Secondary" 3 "Higher-Secondary" ///
4 "University-Degree" 5 "PhD"
```

Note that in this example *EDU* is the name of this combination of labels, that is separate from the name of the variable to which you want to assign this label. For example, you can also give the label the same name as the variable. I prefer to use capital letters on the labels in order to distinguish them from the variable.

2. Then, you have to assign the labels that you have created to the variable:
3. First, you a to define a *label value*:

```
label values education EDU
```

Similarly, you can assign a label also to the variable, by doing:

```
label var education "Education-attainment"
```

And to the entire database:

```
label data "Individuals-with-education-attainment"
```

Similarly to the commands `destring` and `tostring`, you can transform a numerical variable with labels in a string by doing:

```
decode education , gen(edu_string)
```

And vice versa by doing:

```
encode edu_string , gen(edu_num)
```

Note that both commands do not support the option `replace`!

Question

What is the main difference between the commands `tostring` and `decode`?

5 Including dummies and interactions in regressions in Stata

As seen, doing linear regression in Stata is quite easy. You simply need to type:

```
regress y x1 x2 x3
```

where `y`, `x1`, `x2` and `x3` are variable names.

Stata automatically consider the first variable as the dependent variable, while all the remaining variables are independent variables (regressors).

By default, Stata accepts only numerical variables and includes them linearly. If you want to include a dummy for each value of a categorical variable, you have to insert it as:

```
regress y i.x1 x2 x3
```

If you want to include a categorical (string) variable, you have to encode it first:

```
encode x1 , ge(x1_num)  
regress y i.x1_num x2 x3
```

If you need to include in your regression an interaction between two categorical variables, you can also directly include it in your equation by doing:

```
encode x1 , ge(x1_num)  
regress y x1_num##x2
```

In this case, Stata takes a dummy for each value of the variables, plus a dummy for each value of the interactions (excluding baseline values). In other words, it is equivalent to specifying:

```
encode x1 , ge(x1_num)  
regress y i.x1_num##i.x2
```

If you do not feel comfortable including interactions in this way, you can obviously manually generate a dummy for each combination of values:

```

forvalues i = 1(1)5{
    forvalues j=1(1)4{
        ge dummy_i`i' _j`j' = (x1_num == `i' & x2 == `j')
    }
}
regress y i.x1_num i.x2 dummy_i*

```

6 Exporting regression results

Stata 18 introduced the command `etable` that allows to produce of estimation tables. Once you estimate a regression, you need to save the results with the command `est store`:

```

regress y i.x1 x2 x3
est store myregr

```

And then, you can export with the command `etable`:

```

etable , estimates(myregr) export(filename , replace)
* or
etable , estimates(myregr) showstars showstarsnote ///
    title("My-Regression-Table") export(mydoc.pdf , replace)

```

where *filename* can be a Word file (.docx), an Excel file (.xls, .xlsx) or L^AT_EX(.tex).

In empirical applications, you may want to compare how your coefficient of interest changes with the inclusion of additional covariates. To do this, you may want to compare different specifications in the same table. You can do this, by doing:

```

// regression 1
regress y i.x1
est store myregr1
// regression 2
regress y i.x1 x2
est store myregr2
// regression 3
regress y i.x1 x2 x3
est store myregr3
etable , estimates(myregr1 myregr2 myregr3) showstars showstarsnote ///
    title("Regressions-Table") export(myregr_table.pdf , replace)

```

7 Producing descriptive statistics table

Stata 18 also introduced the command `dtable` that allows to produce of descriptive statistics tables. The syntax is:

```

dtable varlist

```

```
dtable x1  
dtable x1 i.x2
```

`dtable` produce a descriptive statistics table. You can export it by doing:

```
dtable varlist , export(mytable.docx)  
dtable varlist , export(mytable.docx , replace)
```

The command `dtable` allows also to construct descriptive table *by* some groups. For example:

```
dtable varlist , by(groupvar)  
dtable varlist , by(groupvar , nototal tests)
```

Applied Industrial Organization

Lect 5

Gianluca Mazzarella

February-May 2026

1 From long to wide and vice-versa – the command reshape

As we have seen, in the estimates of a demand equation you may need information on prices and quantities of other products (complements, substitutes). This information can be in the same line of the same row. For example you can have variables such as: `price_prodA`, `quant_prodA`, `price_prodB`, `quant_prodB`, etc. Or they may be contained in the form:

product	price	quantities
A	10	100
B	8	120

In the first case, we call it database in *wide* format. In the second, in *long* format. You can easily move from one format to the other with the command `reshape`, by using:

- `reshape long` if you want to pivot your data from wide to long
- `reshape wide` if you want to pivot your data from long to wide

The command `reshape` requires three arguments:

1. One or more variables (named *stub*) that you want to pivot.
2. A (set of) variable(s) named “i” that identifies your group of observation. In other words, the dimension that you want to *fix*.
3. A variable named *j* that contains the dimension that you want to pivot. In the case of `reshape wide` the variable already exists and it will be deleted. While, in the case of `reshape long`, the variable does not exist and it will be generated.

The syntax is the following:

```
reshape wide stub_var , i(i_varname) j(j_varname)
reshape long stub_var , i(i_varname) j(j_varname)
```

Reshape wide: how it works?

- The stub variables already exist.
- Stata creates new variables named as the name of the stub variables plus each values contained in the variable j.
- The variable j will be deleted

Reshape long: how it works?

- The stub variables are the prefix of a group of variables.
- Stata creates a new variable named as the stub variable.
- Then Stata generates a new variable j containing the “difference” between the original variable name and the stub name.